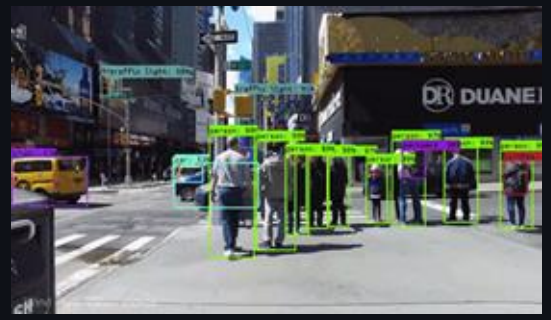
PÓS-GRADUAÇÃO

Computer Vision & IA Generativa



Aula 1 · Introdução à visão por computador + primeiros passos em OpenCV

Duração: ~2h · Teoria + demonstrações práticas em Python

Miguel Ribeiro

PLANO DA AULA

O que vamos ver hoje

Parte 1 — Fundamentos

O que é computer vision, como as máquinas veem imagens e onde isto se aplica no mundo real.

Parte 2 — As tarefas

Panorama: classificação, deteção, segmentação, OCR, reconhecimento facial, pose, geração de imagem e vídeo.

Parte 3 — OpenCV

A biblioteca-base do curso: abrir imagens, recortar, desenhar e ler vídeo da webcam, com código.

Parte 4 — Prática

Erros comuns, um mini-exercício e o que preparar para a próxima aula.

DEFINIÇÃO

O que é Computer Vision?

Computer Vision (visão por computador) é a área da inteligência artificial que dá às máquinas a capacidade de **extrair informação e compreender o mundo a partir de imagens e vídeo.**

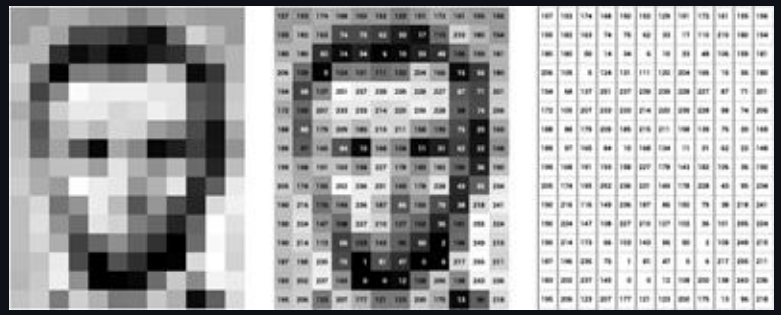
Onde o olho humano vê uma cena, o computador recebe apenas uma matriz de números. O objetivo da CV é transformar esses números em decisões: o que está na imagem, onde está e o que fazer com isso.

- **Perceber** — detetar e reconhecer
- **Interpretar** — dar significado
- **Agir** — decidir e responder

O QUE A MÁQUINA "VÊ"

120	122	118	90	40	38	42	60
125	130	124	88	35	30	45	66
118	126	200	210	205	33	40	70
110	120	205	235	208	41	44	72

Cada número = intensidade de um pixel (0–255). É só disto que o algoritmo dispõe.

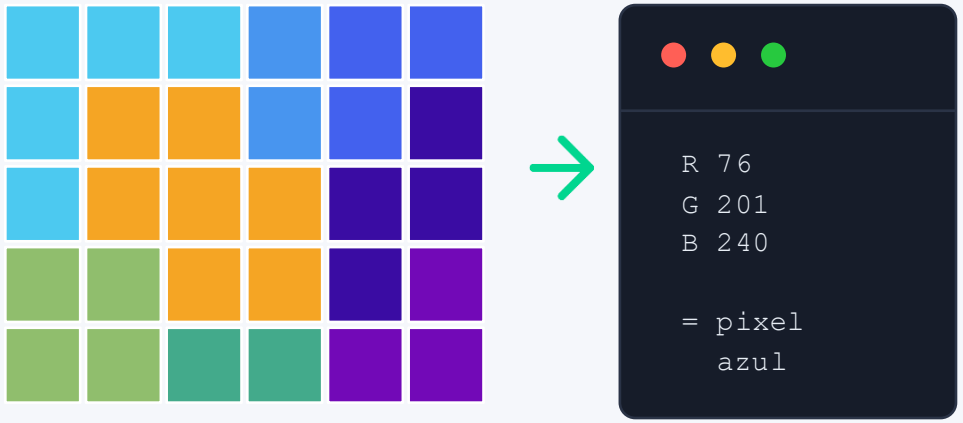


FUNDAMENTOS

Como as máquinas "veem" uma imagem

- Uma imagem digital é uma grelha (matriz) de pixels — pequenos quadrados de cor.
- Cada pixel guarda um ou mais valores numéricos, tipicamente entre 0 e 255.
- Imagem em tons de cinzento → 1 valor por pixel (intensidade).
- Imagem a cores → 3 valores por pixel: os canais Vermelho, Verde e Azul (RGB).
- A "resolução" (ex.: 1920×1080) é o número de pixels na largura × altura.

Imagem → grelha de pixels → números



Ampliar uma imagem o suficiente revela a estrutura: tudo é feito de pixels, e cada pixel é feito de números.

MOTIVAÇÃO

Porque é que Computer Vision importa

~80%

dos dados que geramos são visuais — imagens e vídeo pedem ferramentas automáticas para os interpretar.

24/7

os sistemas de visão não se cansam nem se distraem: inspecionam, monitorizam e medem sem parar.

ms

decisões em milissegundos — carros, drones e linhas de produção reagem mais depressa que um humano.

A visão por computador saiu dos laboratórios

Nos últimos anos, o deep learning e o hardware acessível (GPUs) tornaram a CV prática e barata. Hoje corre no telemóvel que desbloqueia com a sua cara, no carro que trava sozinho, no hospital que analisa radiografias e na app que gera imagens a partir de texto. É uma das tecnologias com impacto mais visível no dia-a-dia — e a base de grande parte da IA generativa ligada a imagem que vamos explorar neste curso.

PANORAMA

As grandes tarefas de Computer Vision

Vamos ver cada uma a seguir. A maioria dos problemas reais combina várias.

Classificação

que objeto é?

Deteção

onde estão os objetos?

Localização

posição exata na imagem

Segmentação

que pixels pertencem a quê

OCR

ler texto em imagens

Reconh. facial

quem é esta pessoa?

Pose estimation

postura do corpo

Tracking

seguir ao longo do vídeo

Text-to-Image

gerar imagem de texto

Image-to-Video

animar imagens

Análise / perceção

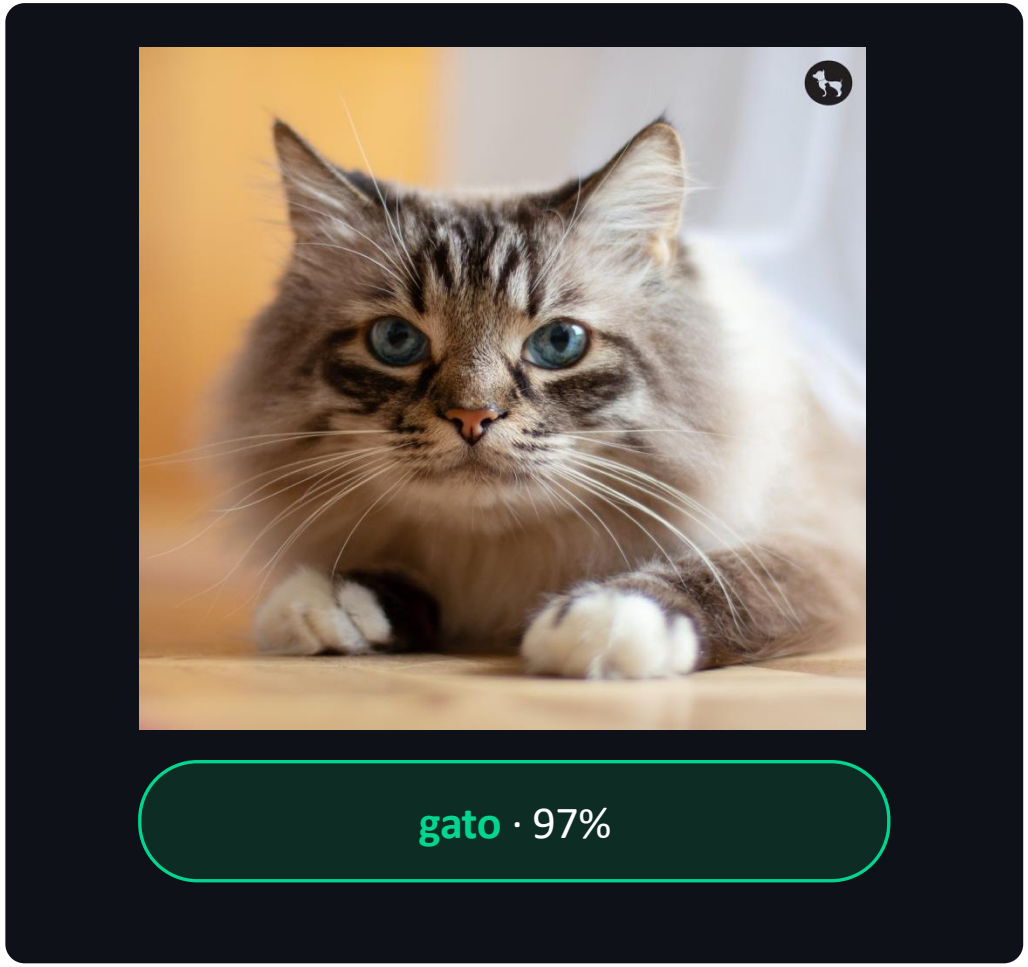
Geração (IA generativa)

TAREFA 1

Classificação de imagem

Responde à pergunta mais simples: "o que é que esta imagem mostra?" — atribui um rótulo (ou vários) à imagem inteira.

- Entrada: uma imagem. Saída: uma etiqueta + confiança (ex.: "gato" 0.97).
- Não diz onde está o objeto, apenas que ele existe na imagem.
- Base histórica do deep learning em visão (ImageNet, CNNs).
- Usos: filtrar conteúdo, organizar fotos, controlo de qualidade.



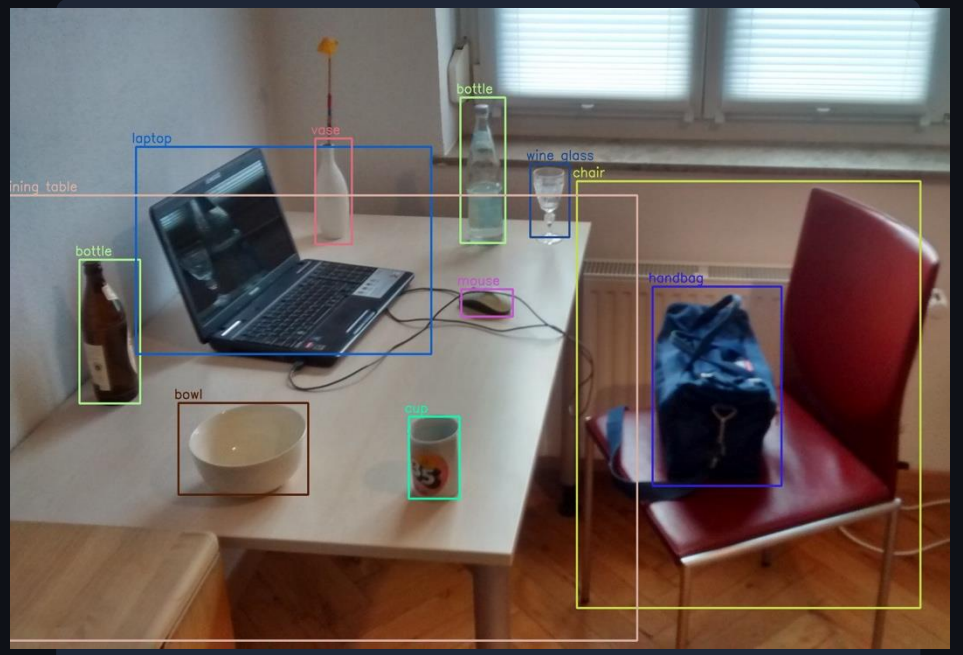
TAREFA 2

Deteção de objetos

Encontra vários objetos numa imagem e devolve, para cada um, uma "bounding box" (retângulo) + a classe.

- Saída: lista de retângulos com coordenadas + rótulo + confiança.
- Deteta múltiplos objetos em simultâneo, mesmo sobrepostos.
- Modelos conhecidos: YOLO, SSD, Faster R-CNN.
- Usos: contar pessoas, vigilância, condução autónoma, retalho.

bounding boxes



Cada objeto → um retângulo (x, y, largura, altura) + a sua classe.

TAREFA 3

Localização

Determina a posição exata de um objeto na imagem — normalmente um único objeto principal, com a sua bounding box.

- Classificação diz "há um gato"; localização diz "está aqui".
- Muitas vezes combinada com classificação → "classificação + localização".
- Coordenadas em pixels: canto superior-esquerdo (x, y) + tamanho.
- É a ponte conceptual entre classificar e detetar vários objetos.



A saída é um único retângulo bem posicionado sobre o objeto de interesse.

TAREFA 4

Segmentação

Em vez de um retângulo, classifica cada pixel. Divide a imagem em regiões com significado, ao nível do contorno.

- Segmentação semântica: cada pixel recebe uma classe (estrada, céu, pessoa).
- Segmentação de instâncias: distingue objetos individuais da mesma classe.
- Muito mais precisa que uma bounding box — segue a forma real.
- Usos: imagem médica, edição de fotos, carros autónomos.



TAREFA 5

OCR — reconhecimento de texto

Optical Character Recognition: localiza e converte texto presente em imagens em texto editável e pesquisável.

- Deteta a região do texto e depois reconhece os caracteres.
- Funciona em documentos, matrículas, faturas, placas, legendas.
- Ferramentas: Tesseract, EasyOCR, serviços cloud, PaddleOCR.
- Usos: digitalizar papel, ler matrículas, automatizar faturação.

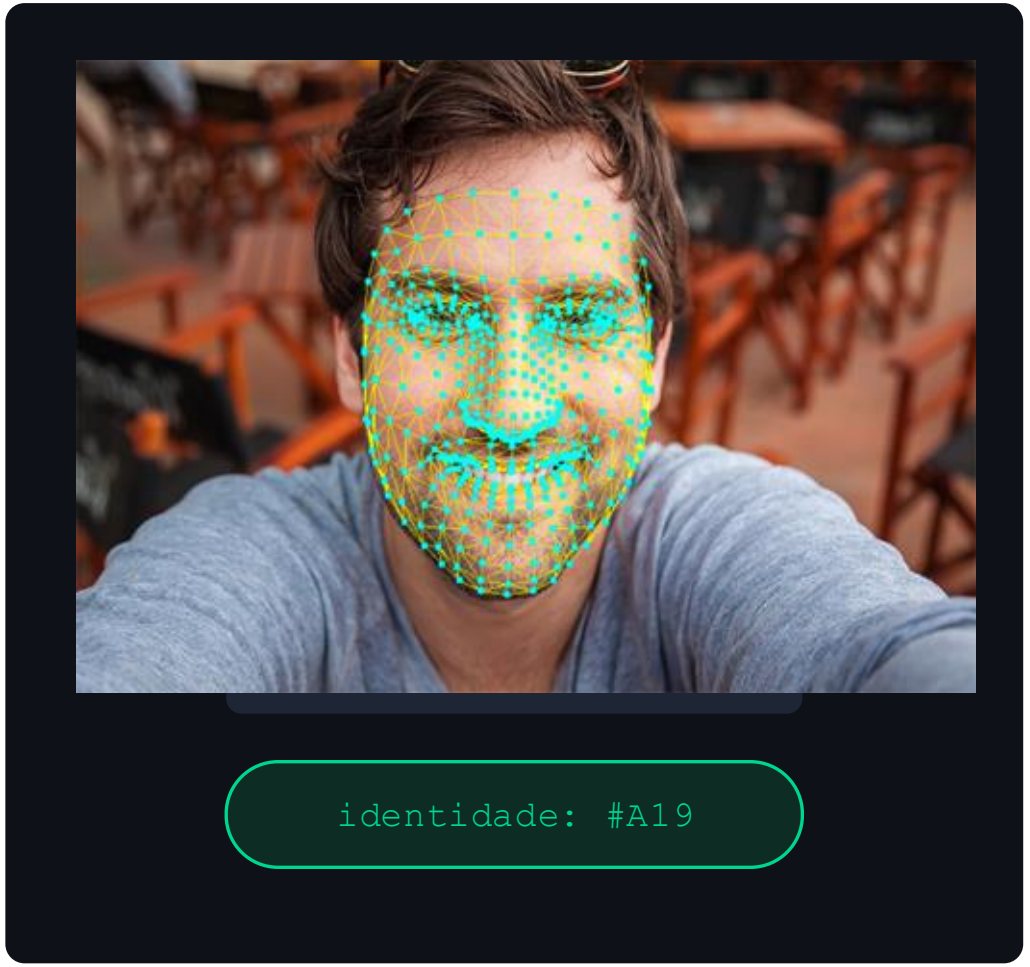


TAREFA 6

Reconhecimento facial

Três problemas distintos: detetar rostos, reconhecer quem é, e verificar se duas caras são a mesma pessoa.

- Deteção de rosto: onde estão as caras na imagem.
- Reconhecimento: a que identidade corresponde cada rosto.
- Verificação: estas duas fotos são da mesma pessoa? (1:1).
- Usos: desbloquear telemóvel, controlo de acessos. Atenção à privacidade e ética.



TAREFA 7

Pose estimation

Estima a postura do corpo detetando pontos-chave (juntas: ombros, cotovelos, joelhos) e ligando-os num esqueleto.

- Saída: coordenadas de keypoints + ligações entre eles.
- Funciona para corpo, mãos e rosto; 2D ou 3D.
- Ferramentas: MediaPipe, OpenPose, MoveNet.
- Usos: fitness, fisioterapia, jogos, linguagem gestual, ergonomia.

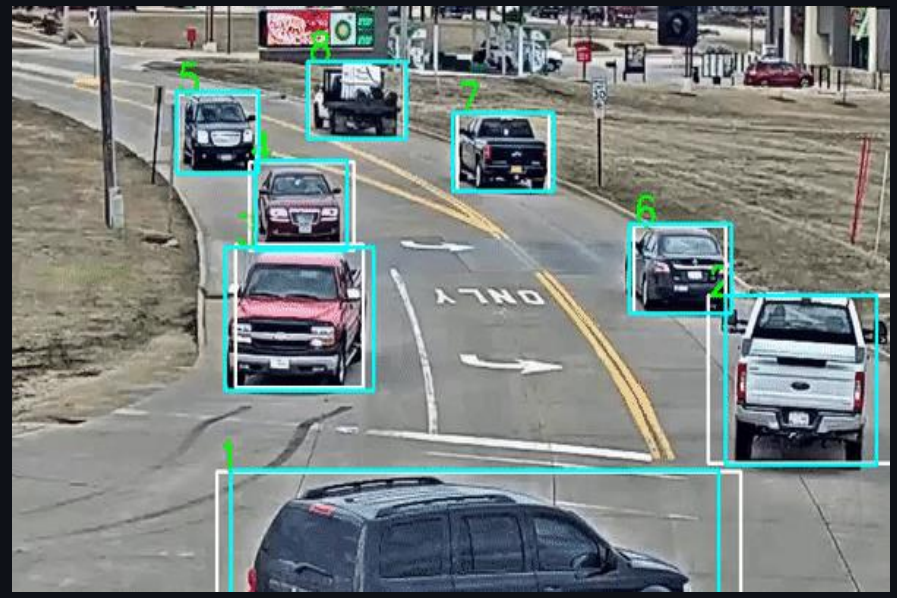


TAREFA 8

Tracking (seguimento)

Deteção acontece numa imagem; tracking segue o mesmo objeto ao longo dos frames de um vídeo, mantendo-lhe uma identidade.

- Atribui um ID estável a cada objeto e segue-o no tempo.
- Lida com oclusões, entradas e saídas de cena.
- Combina-se com deteção: "detect + track" (ex.: ByteTrack, DeepSORT).
- Usos: contagem de tráfego, análise desportiva, robótica.



O mesmo objeto mantém o ID #7 frame após frame.

SEGUNDO GRANDE BLOCO

IA Generativa ligada a imagem

Até aqui, os modelos analisam imagens. Agora invertemos: modelos que criam imagens e vídeo novos.

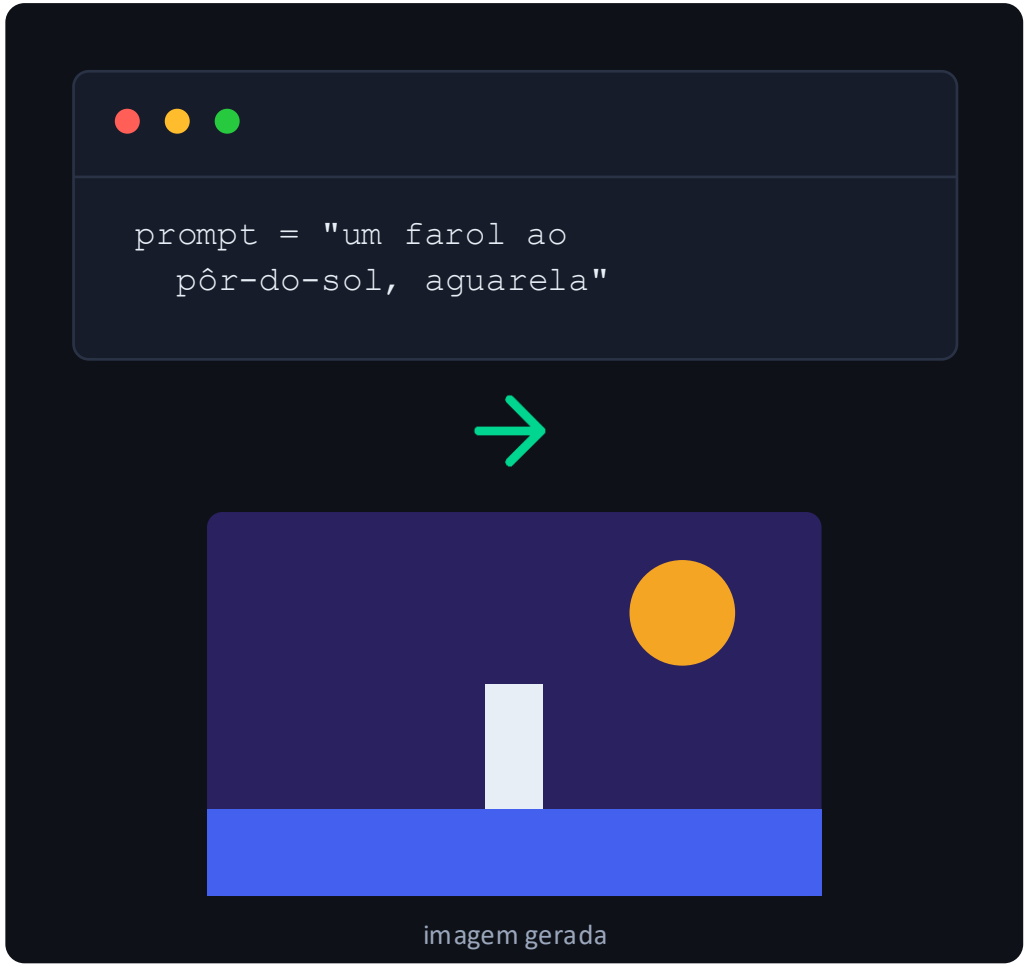
- **Text-to-Image** texto → imagem
- **Image editing** inpainting, variações
- **Image / Text-to-Video** imagem → vídeo

GERAÇÃO 1

Text-to-Image

Escreve-se uma descrição em texto (o "prompt") e o modelo gera uma imagem nova que corresponde a essa descrição.

- Baseado sobretudo em modelos de difusão (diffusion models).
- Aprende com enormes conjuntos de imagens + legendas.
- Exemplos: Stable Diffusion, DALL·E, Midjourney, Flux.
- Usos: design, publicidade, protótipos, ilustração, arte conceptual.
- Questões: direitos de autor, enviesamentos, conteúdo falso.



GERAÇÃO 2

Edição e transformação de imagem

Os mesmos modelos generativos podem receber uma imagem de partida e modificá-la de forma controlada.

Image-to-image

Reinterpreta uma imagem seguindo um novo prompt ou estilo, mantendo a composição.

Inpainting

Substitui apenas uma região selecionada — remover ou acrescentar objetos.

Outpainting

Estende a imagem para além das suas margens, inventando o que estaria à volta.

Super-resolução

Aumenta a resolução e o detalhe de imagens pequenas ou desfocadas.

GERAÇÃO 2

Edição e transformação de imagem

Image-to-image

Reinterpreta uma imagem seguindo um novo prompt ou estilo, mantendo a composição.



GERAÇÃO 2

Edição e transformação de imagem

Inpainting

Substitui apenas uma região selecionada — remover ou acrescentar objetos.



GERAÇÃO 2

Edição e transformação de imagem

Outpainting

Estende a imagem para além das suas margens, inventando o que estaria à volta.



GERAÇÃO 2

Edição e transformação de imagem

Super-resolução

Aumenta a resolução e o detalhe de imagens pequenas ou desfocadas.

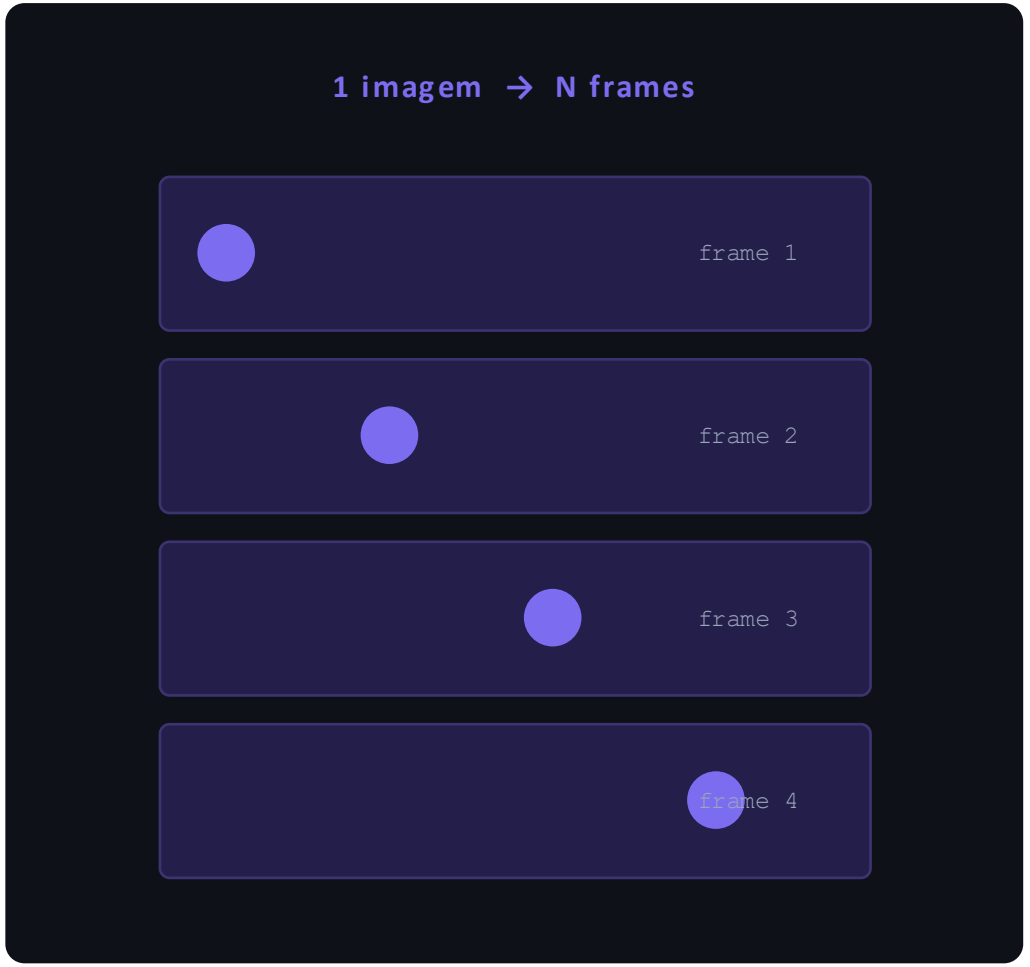


GERAÇÃO 3

Image-to-Video & Text-to-Video

A fronteira mais recente: gerar sequências de vídeo coerentes — a partir de texto, de uma imagem fixa, ou de ambos.

- Um vídeo é uma sequência de imagens (frames) — mais o desafio da consistência temporal.
- O modelo tem de manter objetos coerentes e movimento credível entre frames.
- Exemplos de sistemas: Sora, Runway, Veo, Kling, Stable Video Diffusion.
- Usos: publicidade, cinema, previsualização, redes sociais.
- Muito exigente em computação; qualidade a evoluir rapidamente.



APLICAÇÃO

Casos de uso, por setor

Mobilidade

Condução autónoma, deteção de faixa, sinais e peões, monitorização do condutor.

Saúde

Análise de radiografias e TAC, deteção de tumores, contagem de células.

Retalho

Lojas sem caixas, gestão de stock, análise de fluxo de clientes.

Segurança

Videovigilância inteligente, controlo de acessos, deteção de anomalias.

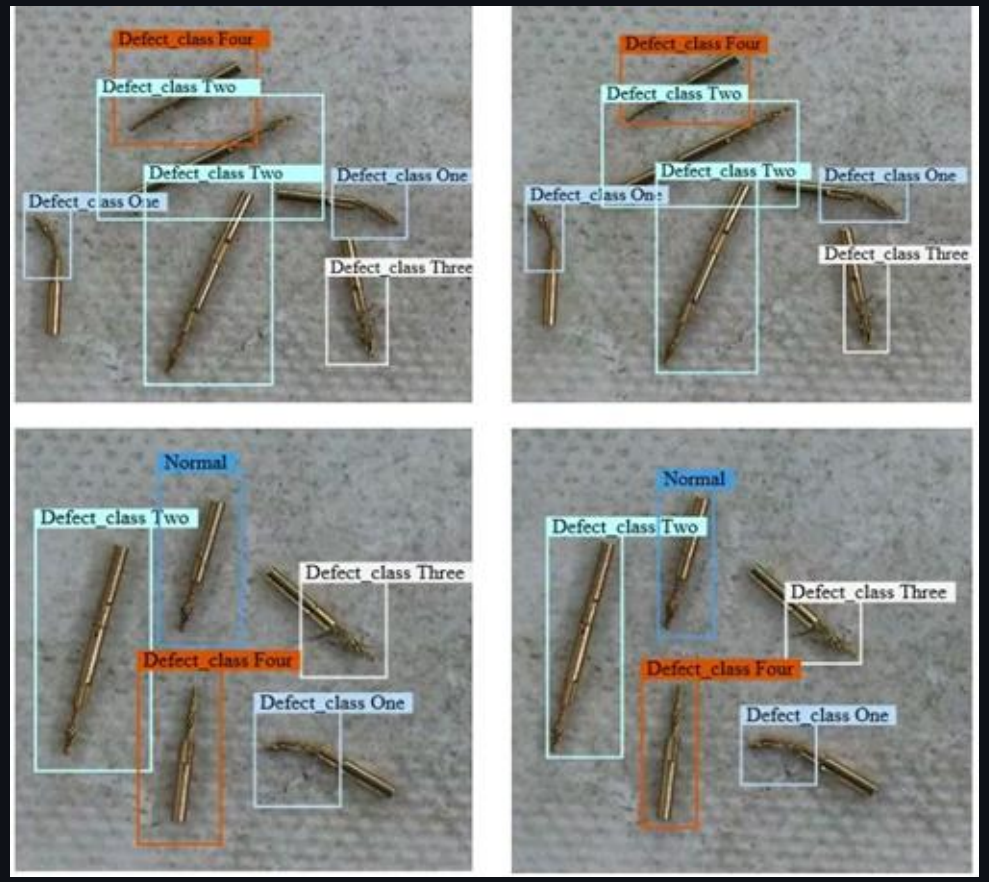
Indústria

Inspeção de qualidade, deteção de defeitos, robótica na linha de montagem.

Consumo

Filtros e efeitos, desbloqueio facial, pesquisa visual, realidade aumentada.

Controlo de qualidade



As aplicações de câmaras inteligentes proporcionam um método escalável para implementar a inspeção visual automatizada e o controlo de qualidade dos processos de produção e das linhas de montagem em fábricas inteligentes.

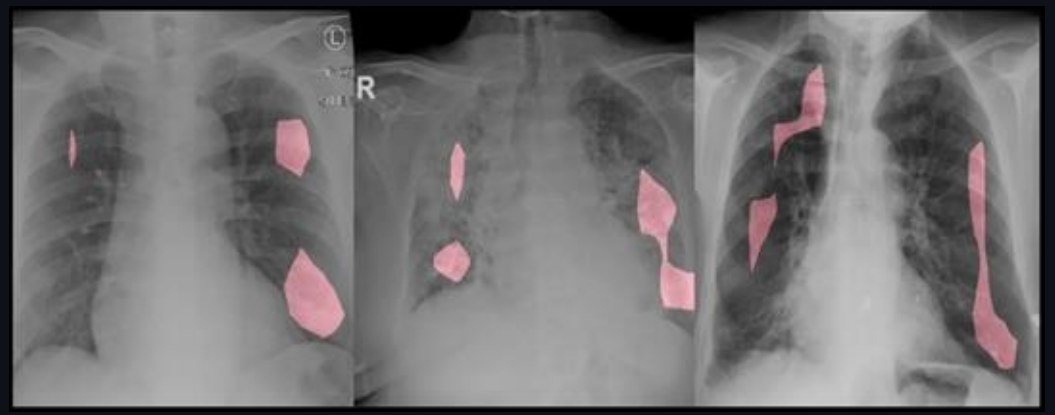
Neste contexto, a aprendizagem profunda utiliza a [deteção de objetos em tempo real para proporcionar resultados superiores (precisão de deteção, velocidade, objetividade e fiabilidade) em comparação com a morosa inspeção manual.

Inspeção de equipamento



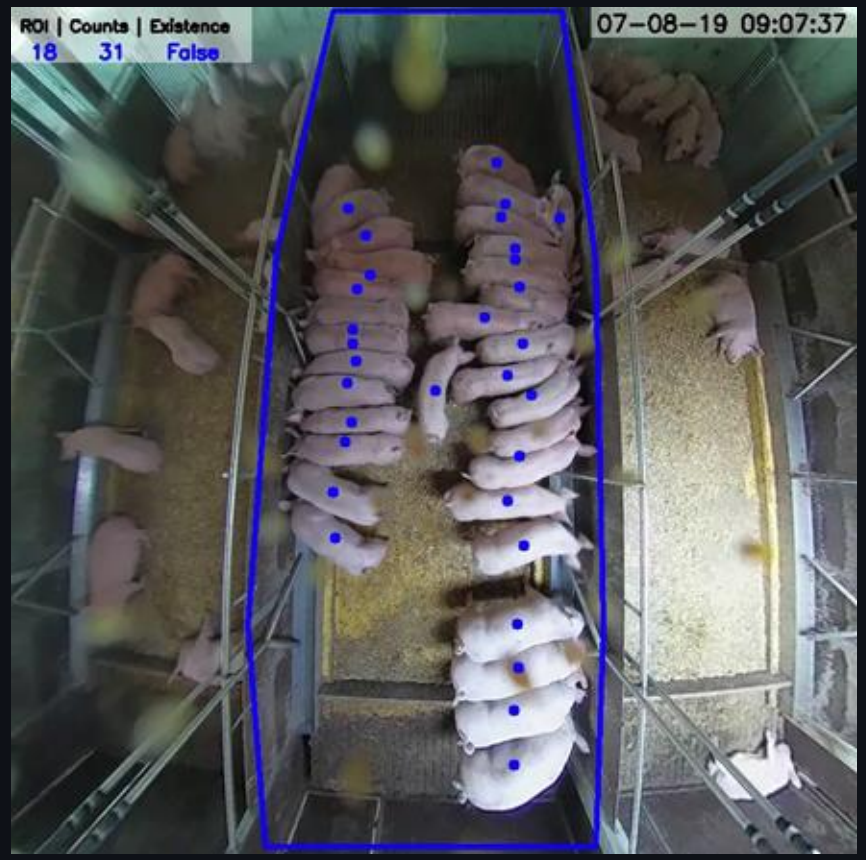
A visão computacional para inspeção visual é uma estratégia fundamental no fabrico inteligente. Os sistemas de inspeção baseados em visão estão também a ganhar popularidade na inspeção automatizada de equipamentos de proteção individual (EPI), como a deteção de máscaras ou a deteção de capacetes. A visão computacional ajuda a monitorizar o cumprimento dos protocolos de segurança em estaleiros de construção ou numa fábrica inteligente.

Saúde



Deteção de cancro
Diagnóstico da COVID-19
Classificação celular
Análise do movimento
Deteção de máscaras
Deteção de tumores
Pontuação da progressão da doença
Cuidados de saúde e reabilitação
Formação em competências médicas

Agricultura



Monitorização de animais

Automatização agrícola

Monitorização das culturas

Deteção da floração

Monitorização das plantações

Deteção de insetos

Deteção de doenças das plantas

Remoção automática de infestantes

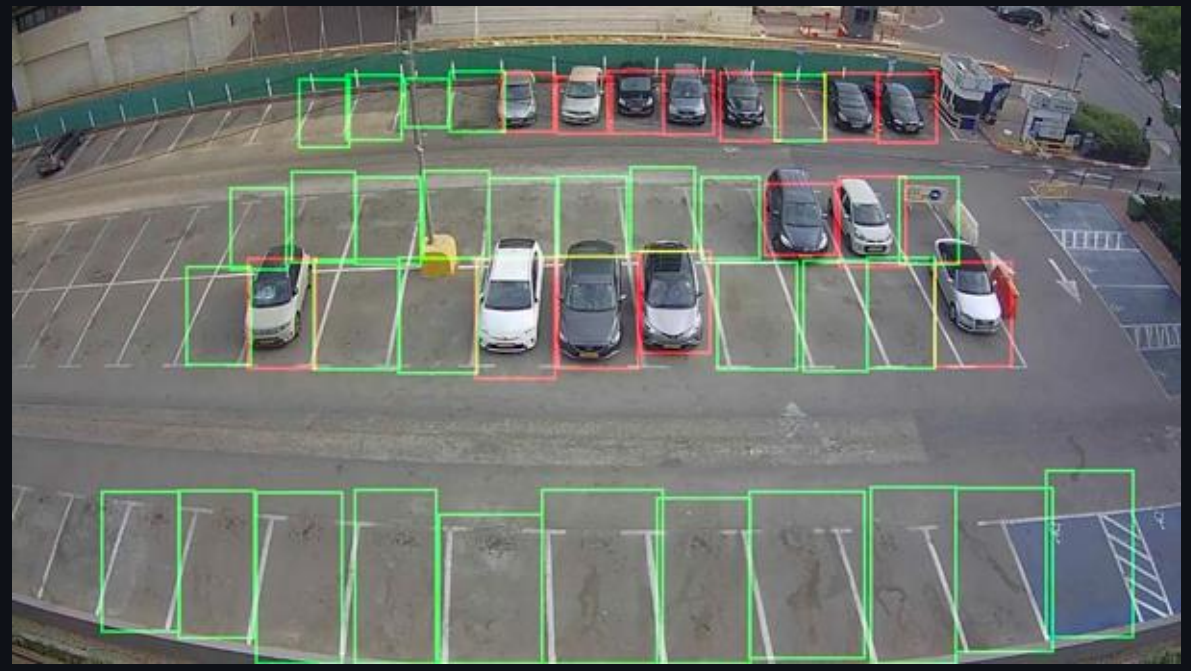
Colheita automática

Avaliação da qualidade dos produtos agrícolas

Gestão da irrigação

Avaliação da produtividade

Transportes



Deteção de infrações de trânsito

Análise do fluxo de tráfego

Deteção da ocupação de lugares de estacionamento

Reconhecimento automático de matrículas (ALPR)

Reidentificação de veículos

Deteção de peões

Deteção de sinais de trânsito

Sistemas de prevenção de colisões

Monitorização do estado das estradas

Avaliação do estado das infraestruturas

Deteção do nível de atenção do condutor

Retalho



Monitorização de clientes

Contagem de pessoas

Deteção de furtos

Análise dos tempos de espera

Distanciamento social

Desporto



Rastreamento da pose dos jogadores
Captura de movimento sem marcadores
Avaliação do desempenho
Rastreamento da pose de vários jogadores
Reconhecimento de golpes
Orientação em tempo real
Análise de equipas desportivas
Rastreamento da bola
Tecnologia da linha de golo
Deteção de eventos desportivos
Geração de melhores momentos

COMO SE TRABALHA

O fluxo típico de um projeto de visão

1

Captar

Obter imagens ou vídeo — câmara, dataset, webcam.

2

Pré-processar

Redimensionar, recortar, normalizar, converter cor.

3

Modelar

Aplicar o algoritmo ou modelo (clássico ou deep learning).

4

Interpretar

Ler os resultados: caixas, máscaras, rótulos.

5

Agir

Usar a informação: alerta, decisão, visualização.

Nesta aula focamo-nos nos passos **Captar e Pré-processar** — a base sobre a qual tudo o resto assenta — usando o OpenCV.

PARTE 2 · PRÁTICA

OpenCV com Python

A caixa de ferramentas onde vamos abrir imagens, recortá-las, desenhar sobre elas e ler vídeo da webcam.

```
import cv2

img = cv2.imread(
    "foto.jpg")
cv2.imshow("w", img)
cv2.waitKey(0)
```

FERRAMENTA

O que é o OpenCV?

OpenCV (Open Source Computer Vision Library) é a biblioteca **open-source mais usada para processamento de imagem e visão por computador.**

- Criada em 2000; hoje mantida por uma vasta comunidade.
- Escrita em C++ e muito otimizada, com interface simples em Python.
- Mais de 2500 algoritmos: filtros, deteção, geometria, vídeo…
- Gratuita e multiplataforma (Windows, macOS, Linux, Android).
- Em Python instala-se com um único comando: pip install opencv-python.



FERRAMENTA

Para que serve o OpenCV

Ler e escrever

Abrir, converter e guardar imagens e vídeo em dezenas de formatos.

Processar imagem

Redimensionar, recortar, rodar, filtrar, ajustar cor e contraste.

Desenhar & anotar

Retângulos, linhas, círculos e texto sobre a imagem.

Trabalhar com vídeo

Ler da webcam ou de ficheiros e processar frame a frame.

Visão "clássica"

Deteção de contornos, cantos, bordas e features sem deep learning.

Integrar IA

Servir de ponte para modelos de deep learning (deteção, faces…).

SETUP

Instalação e primeiro import

È necessário ter o Python instalado. Recomenda-se um ambiente virtual para isolar as dependências do projeto.

1 · Criar e ativar um ambiente virtual

```
python -m venv venv
source venv/bin/activate
# Windows: venv\Scripts\activate
```

2 · Instalar o OpenCV (e o NumPy vem junto)

```
pip install opencv-python numpy
```

3 · Confirmar que funciona

```
import cv2
print(cv2.__version__)
# ex.: 4.10.0
```

Se a versão aparece no terminal, está tudo pronto para começar.

Nota: opencv-python inclui a interface gráfica (imshow). Em servidores sem ecrã usa-se antes opencv-python-headless.

CONCEITO-CHAVE

No OpenCV, uma imagem é um array NumPy

- Ao ler uma imagem, o OpenCV devolve um array NumPy de números.
- A "forma" (shape) é (altura, largura, canais) — repara: altura primeiro!
- Cada valor é um inteiro de 0 a 255 (8 bits por canal).
- Atenção crucial: o OpenCV usa a ordem de canais BGR, não RGB.
- Por isso ao mostrar com outras bibliotecas (ex.: Matplotlib) as cores trocam — é preciso converter.

```
shape = (altura, largura, 3)
```

OpenCV lê nesta ordem:



a maioria das outras usa:




```
# converter quando preciso
rgb = cv2.cvtColor(img,
    cv2.COLOR_BGR2RGB)
```

EXEMPLO 1

Abrir e mostrar uma imagem

```python
import cv2

# ler a imagem do disco (BGR)
img = cv2.imread("foto.jpg")

# abrir uma janela a mostrar a imagem
cv2.imshow("A minha foto", img)

# esperar por uma tecla e fechar
cv2.waitKey(0)
cv2.destroyAllWindows()
```

JANELA



imread devolve None se o caminho estiver errado — verifica sempre. waitKey(0) espera indefinidamente; um número mostra durante N ms.

EXEMPLO 2

Propriedades da imagem

```python
import cv2
img = cv2.imread("foto.jpg")

# dimensoes e tipo
print(img.shape)    # (altura, largura, canais)
print(img.dtype)    # uint8  -> valores 0..255

h, w = img.shape[:2]
print("Largura:", w, "Altura:", h)

# valor de UM pixel (linha 0, coluna 0)
```

A imagem é indexada como img[linha, coluna] = img[y, x]. Primeiro a linha (y), depois a coluna (x) — o contrário da intuição.

CONSOLA

```
(1080, 1920, 3)
uint8
Largura: 1920 Altura: 1080
[ 76 201 240 ]
```

EXEMPLO 3

Recortar (crop) uma região

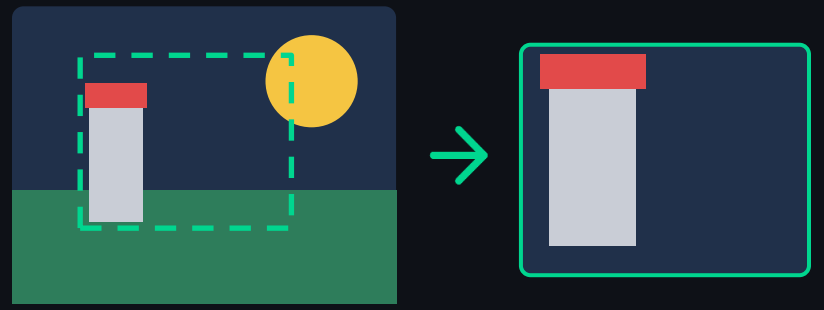
```python
import cv2
img = cv2.imread("foto.jpg")

# um crop e apenas "cortar" o array NumPy:
# img[ y1:y2 , x1:x2 ]
recorte = img[100:400, 200:600]

cv2.imshow("Recorte", recorte)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Não há função "crop": usa-se slicing do NumPy. As coordenadas são [y_inicio:y_fim, x_inicio:x_fim].

ANTES → DEPOIS



`img[100:400, 200:600]`

Guardar é igualmente simples:

```
cv2.imwrite("recorte.jpg", recorte)
```

EXEMPLO 4

Redimensionar (resize)

```python
import cv2
img = cv2.imread("foto.jpg")

# tamanho fixo (largura, altura)
peq = cv2.resize(img, (320, 240))

# por fator de escala (metade)
metade = cv2.resize(img, None,
                    fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)

print(peq.shape)    # (240, 320, 3)
```

Cuidado: em resize o tamanho é (largura, altura) — o inverso de shape. Redimensionar é comum antes de alimentar um modelo.



EXEMPLO 5

Desenhar um retângulo

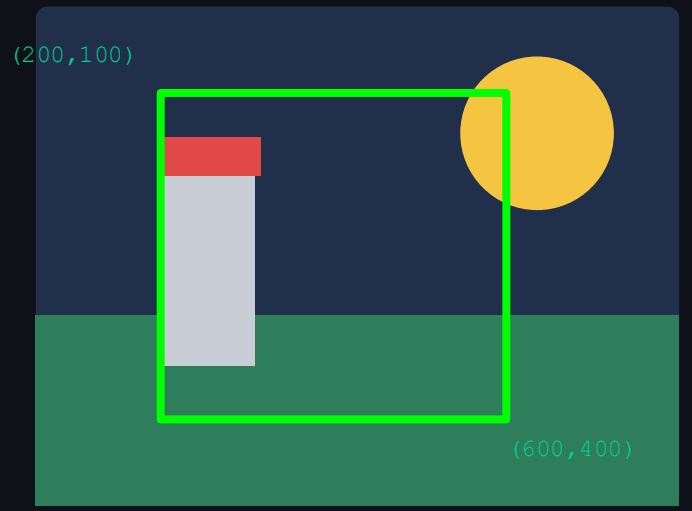
```
import cv2
img = cv2.imread("foto.jpg")

# cv2.rectangle(img, canto1, canto2,
#               cor_BGR, espessura)
cv2.rectangle(img, (200, 100),
              (600, 400),
              (0, 255, 0), 3)

cv2.imshow("Com caixa", img)
cv2.waitKey(0)
```

A cor vem em BGR: (0,255,0) é verde. Espessura -1 preenche o retângulo. É exatamente assim que se desenham bounding boxes de deteção.

BOUNDING BOX



cor = (0, 255, 0) → verde

EXEMPLO 6

Escrever texto na imagem

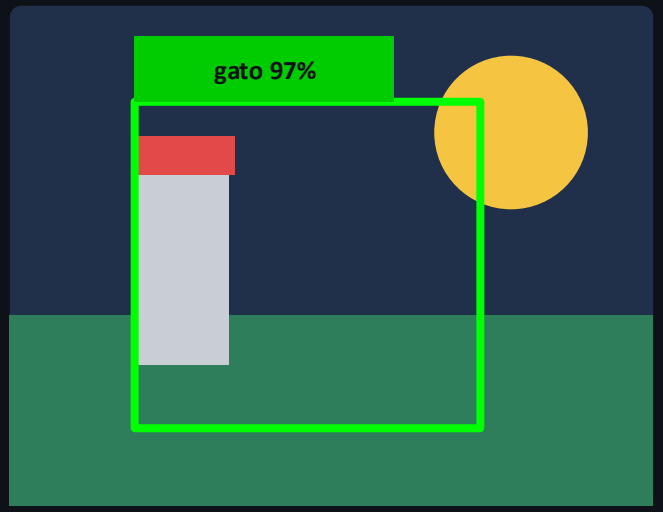
```python
import cv2
img = cv2.imread("foto.jpg")

fonte = cv2.FONT_HERSHEY_SIMPLEX
# cv2.putText(img, texto, posicao,
#   fonte, escala, cor, espessura)
cv2.putText(img, "gato 97%",
            (210, 90), fonte,
            1.0, (0, 255, 0), 2)

cv2.imshow("Rotulo", img)
```

A posição é o canto inferior-esquerdo do texto. Combinado com rectangle, obténs uma caixa com etiqueta — o visual clássico dos detetores.

CAIXA + ETIQUETA



rectangle + putText

EXEMPLO 7

Linhas e círculos

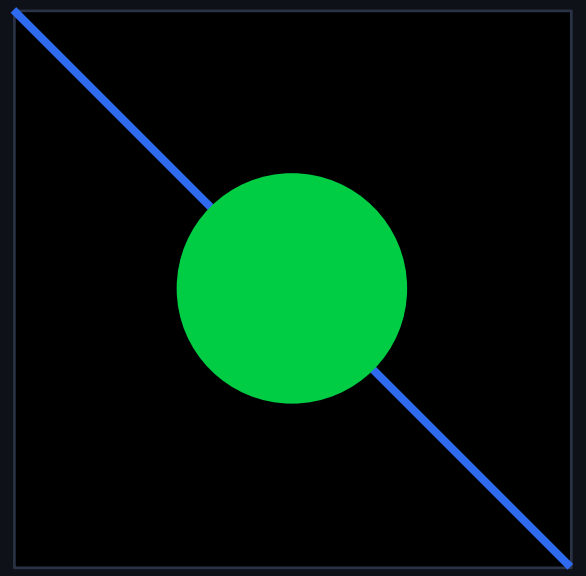
```python
import cv2, numpy as np
# tela preta 400x400
img = np.zeros((400, 400, 3), np.uint8)

# linha: cv2.line(img, p1, p2, cor, esp)
cv2.line(img, (0,0), (400,400),
         (255,0,0), 3)

# circulo: (img, centro, raio, cor, esp)
cv2.circle(img, (200,200), 80,
           (0,255,0), -1)    # -1 = cheio
```

np.zeros cria uma imagem preta do zero. Estas primitivas (linha, círculo, retângulo, texto) chegam para anotar quase qualquer resultado.

TELA 400×400



espessura -1 → preenchido

EXEMPLO 8

Converter cores (cinzento)

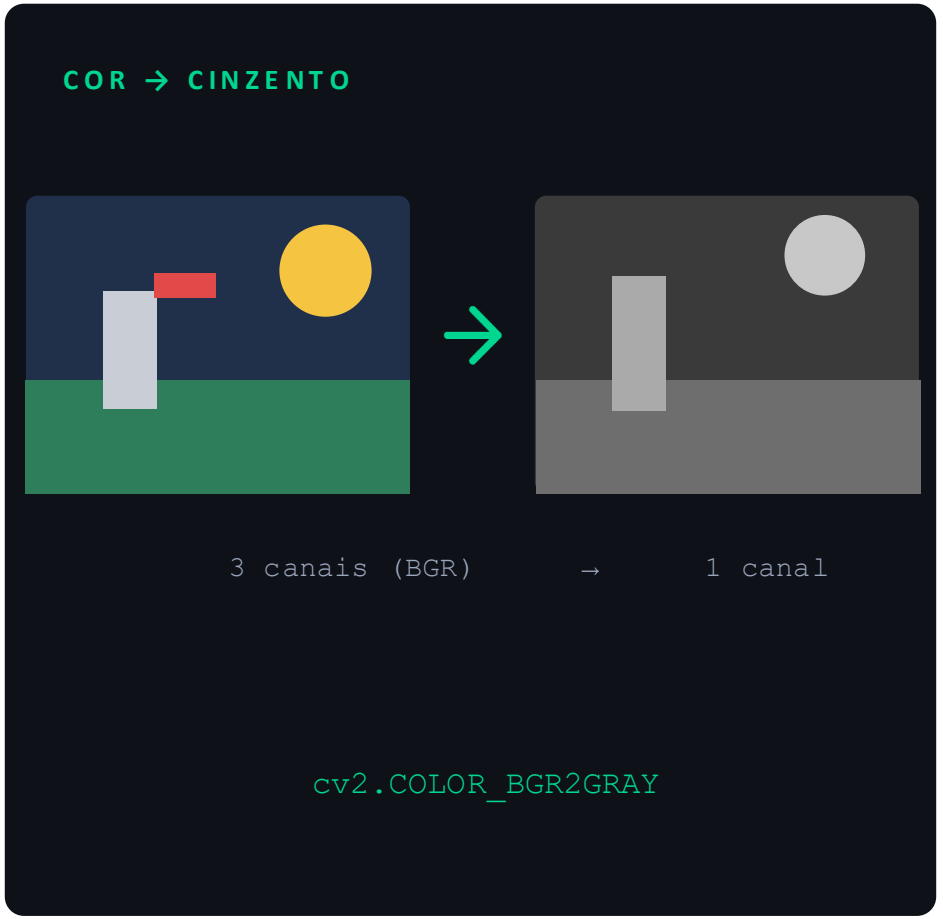
```python
import cv2
img = cv2.imread("foto.jpg")

# BGR -> tons de cinzento
cinza = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)

print(cinza.shape)  # (h, w) -> 1 canal

# BGR -> RGB (para mostrar noutras libs)
rgb = cv2.cvtColor(img,cv2.COLOR_BGR2RGB)
```

Converter para cinzento reduz de 3 canais para 1 e é o primeiro passo de muitos algoritmos clássicos (bordas, deteção de faces).



EXEMPLO 9

Guardar uma imagem

```python
import cv2
img = cv2.imread("foto.jpg")

cinza = cv2.cvtColor(img,
            cv2.COLOR_BGR2GRAY)

# escreve no disco; o formato vem
# da extensao do nome do ficheiro
ok = cv2.imwrite("saida.png", cinza)
print("Guardado:", ok)    # True
```

imwrite escolhe o formato pela extensão (.png, .jpg, .webp…) e devolve True/False conforme conseguiu gravar. Fecha o ciclo ler → processar → guardar.



EXEMPLO 10

Abrir e mostrar a webcam

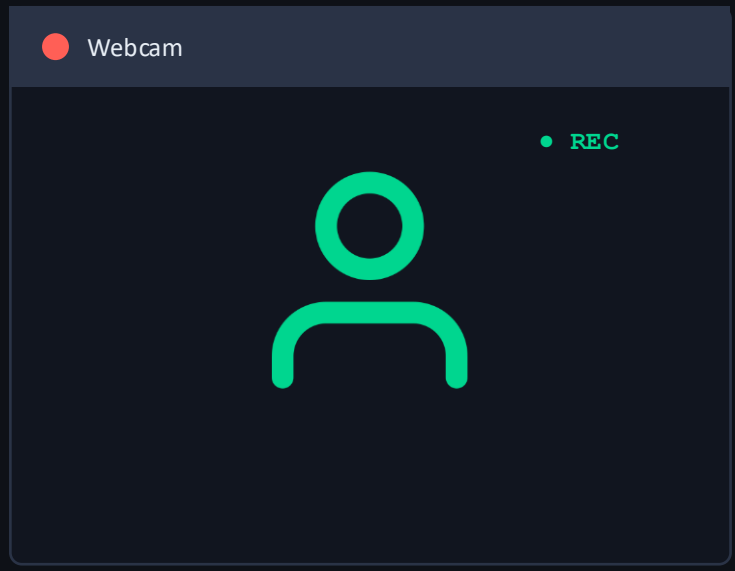
```python
import cv2

cam = cv2.VideoCapture(0)    # 0 = webcam

while True:
    ok, frame = cam.read()    # le 1 frame
    if not ok:
        break
    cv2.imshow("Webcam", frame)
    # sair com a tecla q
    if cv2.waitKey(1) == ord("q"):
        break

cam.release()
cv2.destroyAllWindows()
```

STREAM AO VIVO



Cada volta do ciclo lê e mostra 1 frame → vídeo. Prime "q" para sair.

EXEMPLO 11

Ler um vídeo e processar frames

```python
import cv2
video = cv2.VideoCapture("clip.mp4")

while True:
    ok, frame = video.read()
    if not ok:
        break   # fim do video

    # processar cada frame como imagem
    cinza = cv2.cvtColor(frame,
                cv2.COLOR_BGR2GRAY)
    cv2.imshow("Video", cinza)
    if cv2.waitKey(25) == ord("q"):
        break
video.release(); cv2.destroyAllWindows()
```

VÍDEO = SEQUÊNCIA DE FRAMES



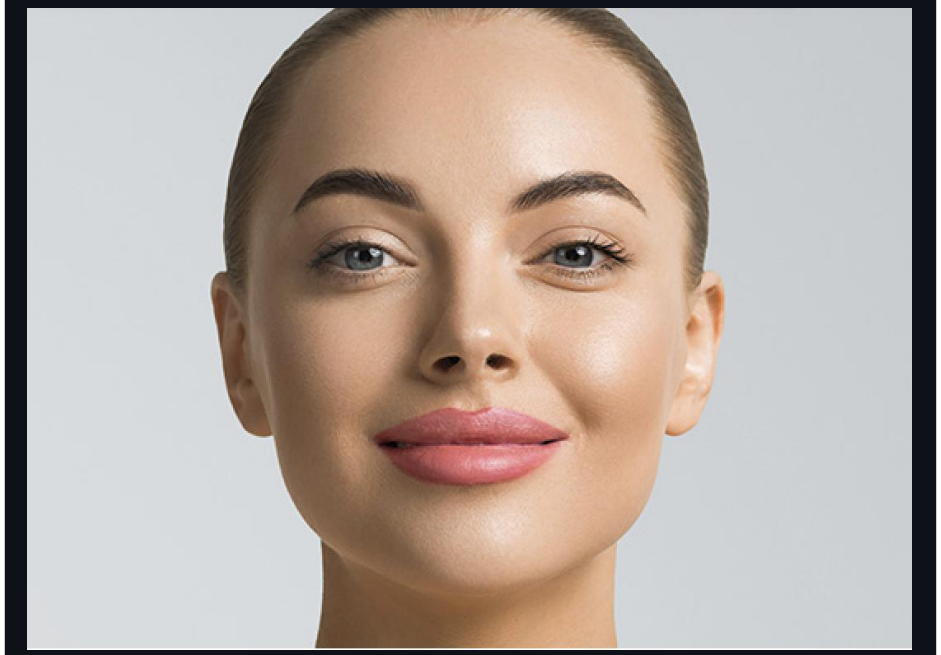
Trata-se cada frame como uma imagem: aqui converte-se para cinzento em tempo real.

EXEMPLO 11

Ler um vídeo e escrever nas frames

```
import cv2
vid = cv2.VideoCapture(0)
while True:
    ret, frame = vid.read()
    if not ret:
        break
    cv2.line(frame, (50, 50), (200, 50), (0, 255, 0)2)
    cv2.circle(frame, (300, 200), 50, (255, 0, 0), 4)
    cv2.rectangle(frame, (400, 400), (500, 500), (0, 0, 255),
thickness= 15)
    cv2.putText(frame, 'Ola ISEG!', (50,
250),cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)
    cv2.imshow('frame', frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
vid.release()
cv2.destroyAllWindows()
```

RESULTADO



EXEMPLO 11

Ler um vídeo e espelhar

```
import cv2

vid = cv2.VideoCapture(0)

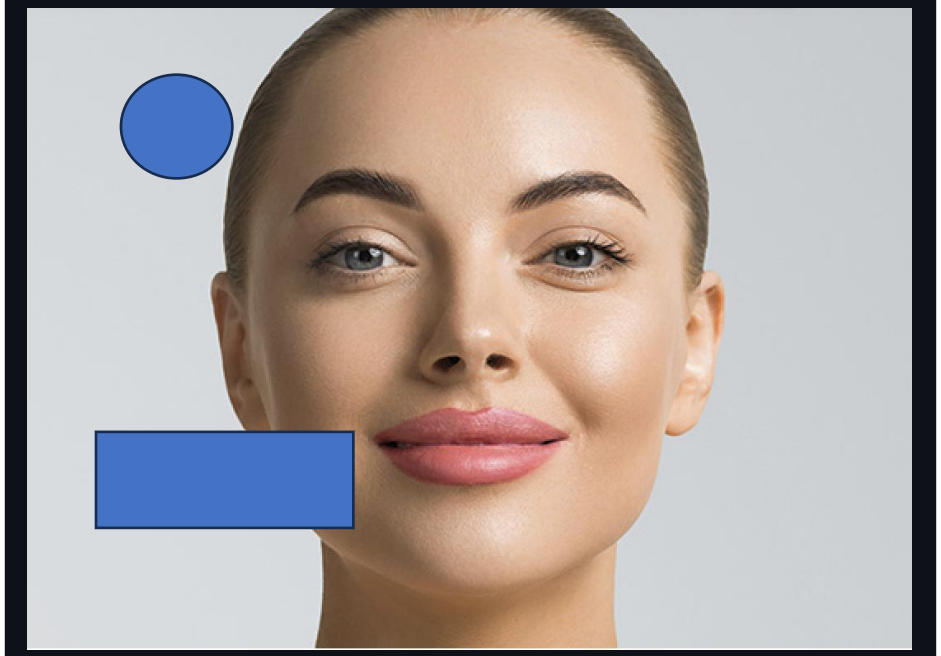
while True:
          ret, frame = vid.read()
          if not ret:
                    break

          # Flip horizontally
          mirror = cv2.flip(frame, 1)

          cv2.imshow('Mirror', mirror)
          if cv2.waitKey(1) & 0xFF == ord('q'):
                    break

vid.release()
cv2.destroyAllWindows()
```

RESULTADO



EXEMPLO 11

Ler um vídeo e ver contornos

```
import cv2

vid = cv2.VideoCapture(0)

while True:
        ret, frame = vid.read()
        if not ret:
                break

        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        edges = cv2.Canny(gray, 100, 200)
        cv2.imshow('Grayscale', gray)
        cv2.imshow('Edges', edges)
        if cv2.waitKey(1) & 0xFF == ord('q'):
                break

vid.release()
cv2.destroyAllWindows()
```

RESULTADO



EXEMPLO 11

Ler um vídeo e detetar rostos

```
import cv2
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
'haarcascade_frontalface_default.xml')
vid = cv2.VideoCapture(0)
while True:
            ret, frame = vid.read()
            if not ret:
                        break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
            #detectMultiScale(image, scaleFactor, minNeighbors) scaleFactor
Controls image scaling between detections, minNeighbors: Controls detection
confidence.Higher value:Fewer false positives, Stricter detection
            faces = face_cascade.detectMultiScale(gray, 1.3, 5)
    for (x, y, w, h) in faces:
                        cv2.rectangle(frame, (x, y), (x+w, y+h), (255, 0, 0),
2)
                        face_region = frame[y:y+h, x:x+w]
                        blurred_face = cv2.GaussianBlur(face_region, (99,
99), 30)
                        frame[y:y+h, x:x+w] = blurred_face
    cv2.imshow('Face Detection', frame)
            if cv2.waitKey(1) & 0xFF == ord('q'):
                        break
vid.release()
cv2.destroyAllWindows()
```

RESULTADO



BOAS PRÁTICAS

Erros comuns a evitar

imread devolve None

Caminho errado → img fica None e rebenta depois. Verifica: if img is None: ...

Trocar BGR e RGB

Cores estranhas (azul/vermelho trocados) ao mostrar noutras libs. Converte com cvtColor.

Ordem (x, y) vs (linha, coluna)

img[y, x] indexa linha depois coluna; resize/pontos usam (x, y). Confundir gera erros.

Esquecer waitKey

Sem cv2.waitKey a janela do imshow não chega a aparecer ou "congela".

Não libertar a câmara

Falta de cam.release() deixa a webcam ocupada para outros programas.

Tamanho invertido no resize

resize espera (largura, altura), o contrário de shape. Fácil de trocar.

MÃOS NA MASSA

Mini-exercício: juntar tudo

Escreva um script que abra uma foto, desenhe uma caixa verde com a etiqueta "objeto" e guarde o resultado.

```
import cv2

img = cv2.imread("foto.jpg")
if img is None:
    raise SystemExit("imagem nao lida")

cv2.rectangle(img, (60, 60), (300, 300),
              (0, 255, 0), 3)
cv2.putText(img, "objeto", (60, 50),
```

Consegue explicar...

- Porque é que a cor (0,255,0) é verde e não vermelho?
- O que acontece se o caminho da foto estiver errado?
- Como mudaria a espessura da caixa para a preencher?
- Como fariaso mesmo em cada frame da webcam?

SÍNTESE

O que levamos desta aula

Computer Vision

- Máquinas que extraem informação de imagens e vídeo.
- Para o computador, uma imagem é uma matriz de números.
- Tarefas: classificar, detetar, segmentar, OCR, faces, pose…

IA Generativa

- Modelos que criam imagens (text-to-image) e as editam.
- Vídeo generativo: imagem/texto → sequência de frames.
- Enorme potencial e questões éticas por resolver.

OpenCV

- A biblioteca-base: imagem = array NumPy em BGR.
- Ler, recortar, redimensionar, desenhar, guardar.
- Vídeo e webcam = processar frame a frame.

FERRAMENTAS

O que é o Gradio?

Biblioteca Python para criar interfaces web à volta de funções — sem escrever HTML nem JavaScript.

- Uma função Python entra; uma interface web sai.
- gr.Interface(fn, inputs, outputs) liga a função aos componentes.
- Componentes prontos: texto, imagem, áudio, webcam, sliders.
- demo.launch() corre localmente; share=True gera um link público.
- Ideal para demonstrar e testar modelos de visão rapidamente.



GRADIO · EXEMPLO 0

A primeira interface

```python
import gradio as gr
def greet(name):
    return ("Hello " + str(name) + "!")
demo = gr.Interface(fn=greet, inputs="text", outputs="text")
demo.launch()
#expose to the outside
#demo.launch(share=True)
```

COMO FUNCIONA

- greet(name) devolve “Hello <name>!”.
- inputs=“text” e outputs=“text” criam caixas de texto.
- gr.Interface liga a função à interface web.
- demo.launch() abre no browser; share=True dá link público.

GRADIO · EXEMPLO 1

Cor para preto e branco

```python
import cv2
import numpy as np
import gradio as gr
def to_grayscale(img):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    # convert back to 3 channels for visualization
    return cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
demo = gr.Interface(
    fn=to_grayscale,
    inputs=gr.Image(type="numpy"),
    outputs=gr.Image(type="numpy"),
    title="Grayscale Converter"
)
demo.launch(share=True)
```

COMO FUNCIONA

- cv2.cvtColor converte BGR para cinzento e volta a 3 canais.
- gr.Image(type="numpy") recebe e devolve arrays.
- title dá um cabeçalho à interface.
- share=True gera um link público para testar.

GRADIO · EXEMPLO 2

Filtros interativos

```python
import cv2
import numpy as np
import gradio as gr
def apply_filter(img, filter_type):
    if filter_type == "Grayscale":
        result = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        result = cv2.cvtColor(result, cv2.COLOR_GRAY2BGR)
    elif filter_type == "Edges":
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        edges = cv2.Canny(gray, 100, 200)
        result = cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)
    elif filter_type == "Blur":
        result = cv2.GaussianBlur(img, (35, 35), 0)
    else:
        result = img
    return result
demo = gr.Interface(
    fn=apply_filter,
    inputs=[gr.Image(type="numpy"), gr.Radio(["Original", "Grayscale", "Edges", "Blur"])],
    outputs=gr.Image(type="numpy"),
    title="Interactive Filters"
)
demo.launch()
```

COMO FUNCIONA

- gr.Radio adiciona um seletor de filtro.
- Cada opção aplica uma operação OpenCV diferente.
- Grayscale, Edges (Canny) e Blur (Gaussian).
- Vários inputs numa lista alimentam a função.

GRADIO · EXEMPLO 3

Deteção de rostos na webcam

```python
import cv2
import gradio as gr
# Load Haar Cascade for face detection
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
)
def detect_faces(frame):
    frame = frame.copy()
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)
    for (x, y, w, h) in faces:
        cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
    return frame
with gr.Blocks() as demo:
    gr.Markdown("## Real-Time Face Detection")
    webcam = gr.Image(sources="webcam", type="numpy")
    output = gr.Image(type="numpy")
    webcam.stream(fn=detect_faces, inputs=webcam, outputs=output)
demo.launch()
```

COMO FUNCIONA

- Haar Cascade deteta rostos em cada frame.
- gr.Blocks() permite um layout personalizado.
- gr.Image(sources=“webcam”) liga à câmara.
- webcam.stream() processa em tempo real.

DLIB

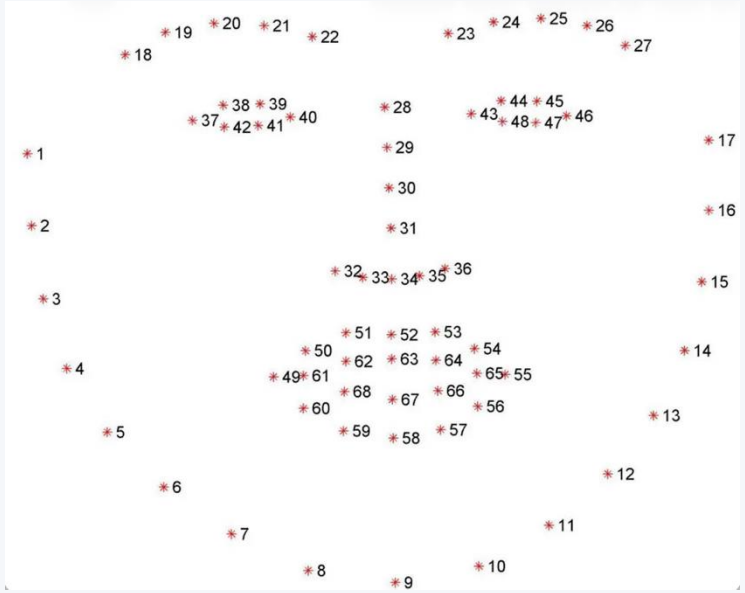
Deteção de Landmarks Faciais

Pip install dlib-bin

Dlib é uma biblioteca de *Machine Learning* e *Computer Vision* que permite detetar e analisar rostos em imagens.
Na deteção de **facial landmarks**, o Dlib identifica pontos de referência específicos da face, como:

Contorno do rosto
Sobrancelhas
Olhos
Nariz
Boca

Um dos modelos mais utilizados deteta **68 pontos faciais**, permitindo representar geometricamente a estrutura do rosto.
Estes landmarks podem ser usados em aplicações como **reconhecimento facial, alinhamento de rostos, deteção de expressões e análise de movimento facial**.

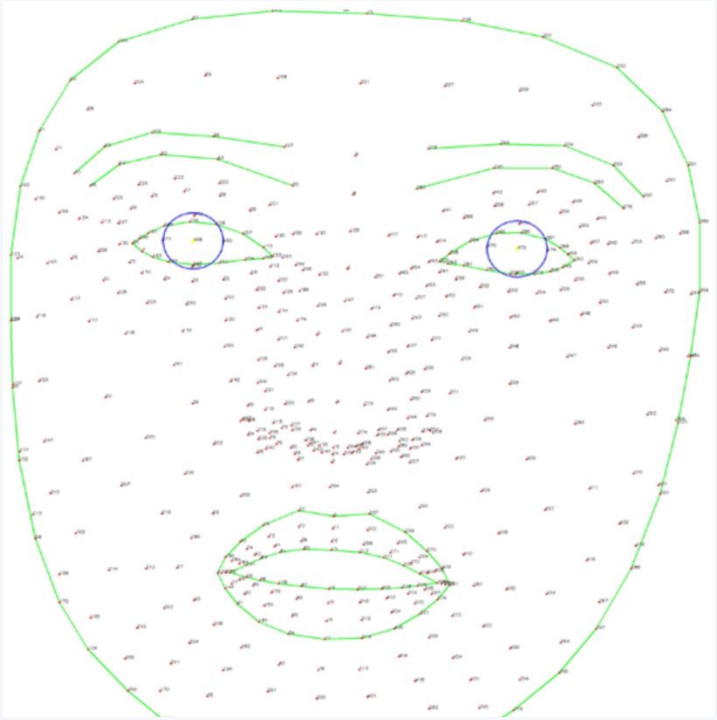


Deteção de Landmarks Faciais

MediaPipe Face Landmarker é uma solução de *Computer Vision* desenvolvida pela Google para detetar e acompanhar pontos de referência faciais em imagens e vídeo.

A partir de um rosto, o modelo identifica uma malha facial composta por centenas de **landmarks 3D**, representando com elevada precisão diferentes regiões da face.
Permite localizar elementos como:
Olhos e sobrancelhas
Nariz
Boca e lábios
Contorno facial
Regiões da testa e bochechas

A deteção pode ser realizada em **tempo real**, tornando o MediaPipe adequado para aplicações interativas.

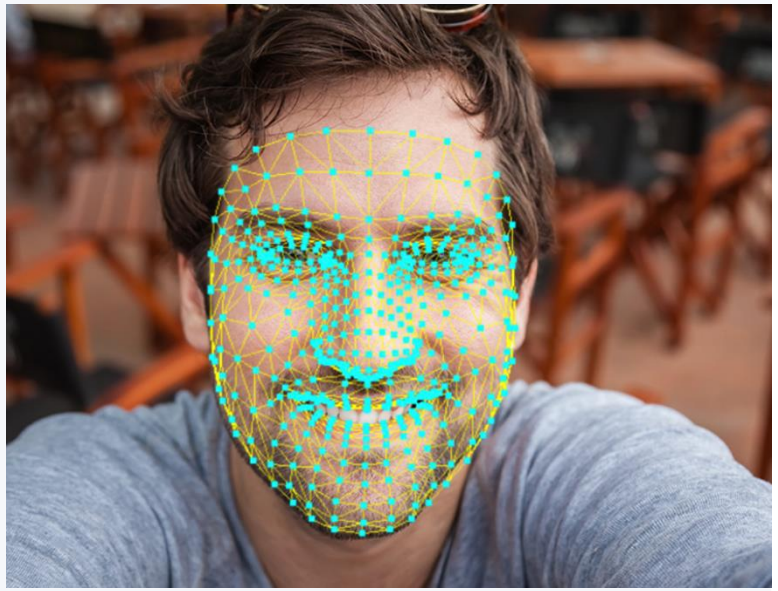


Deteção de Landmarks Faciais

Funcionamento e Aplicações
O MediaPipe utiliza modelos de *Deep Learning* para estimar diretamente a posição dos **landmarks faciais** a partir da imagem.
Cada landmark possui coordenadas **X, Y e Z**, permitindo obter informação não apenas sobre a posição na imagem, mas também sobre a profundidade relativa do rosto.

Principais aplicações:
Seguimento da posição e orientação da cabeça
Reconhecimento de expressões faciais
Filtros e efeitos de Realidade Aumentada
Deteção do movimento dos olhos e da boca
Análise de gestos e comportamento facial

Vantagem: elevada precisão com baixo custo computacional, permitindo processamento em tempo real.



MEDIAPIPE

Deteção de Landmarks mãos

MediaPipe Hands é uma solução de *Computer Vision* para detetar e acompanhar mãos em imagens e vídeo, mesmo em tempo real.

O modelo identifica **21 landmarks por mão**, correspondentes a pontos-chave da palma e dos dedos.
Cada landmark possui coordenadas **X, Y e Z**, permitindo analisar a posição, orientação e movimento da mão.

Principais aplicações:
Reconhecimento de gestos
Controlo de interfaces por movimento
Língua gestual
Realidade Aumentada
Análise do movimento dos dedos e da mão

Vantagem: deteção rápida e precisa, adequada para aplicações em tempo real.

